

Lab Experiment 5: Linear Regression through Gradient Descent

Objective: To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on the Student Performance dataset.

Step 1: Import Libraries

Importing required libraries for data manipulation, visualization, and evaluation.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Step 2: Load the Dataset

Loading the Student Performance dataset (math). We use semicolon as the separator.

```
In [2]: df = pd.read_csv('dataset/student-mat.csv', sep=';')
df.head()
```

```
Out[2]:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	1
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	
3	GP	F	15	U	GT3	T	4	2	health	services	...	
4	GP	F	16	U	GT3	T	3	3	other	other	...	

5 rows x 33 columns

Step 3: Data Preprocessing

Checking for missing values and encoding categorical variables using one-hot encoding. Then, we scale the features to help gradient descent converge quickly.

```
In [3]: # Check missing values
print('Missing values:', df.isna().sum().sum())

# Encode categorical variables
```

```

df_encoded = pd.get_dummies(df, drop_first=True)

# Select features (X) and target (y)
# We want to predict G3 (final grade)
X = df_encoded.drop('G3', axis=1).values
y = df_encoded['G3'].values

# Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print('Training feature shape:', X_train_scaled.shape)

```

Missing values: 0

Training feature shape: (316, 41)

Step 4 & 5: Implement Linear Regression with Gradient Descent

Initializing weights and bias, then iteratively updating them to minimize the Mean Squared Error (MSE) loss.

```

In [4]: def gradient_descent(X, y, learning_rate, iterations):
        n_samples, n_features = X.shape
        # Initialize parameters
        W = np.zeros(n_features)
        b = 0.0
        loss_history = []

        for i in range(iterations):
            # Prediction
            y_pred = np.dot(X, W) + b

            # Calculate error and loss (MSE)
            error = y_pred - y
            loss = np.mean(error ** 2)
            loss_history.append(loss)

            # Gradients
            dW = (2 / n_samples) * np.dot(X.T, error)
            db = (2 / n_samples) * np.sum(error)

            # Update parameters
            W -= learning_rate * dW
            b -= learning_rate * db

        return W, b, loss_history

# Train the model
lr = 0.01
epochs = 1000

```

```
W, b, history = gradient_descent(X_train_scaled, y_train, learning_rate=lr,
print(f'Final Training Loss: {history[-1]:.4f}')
```

Final Training Loss: 2.8266

Step 6: Experiment with Different Learning Rates

Testing different learning rates to see how they affect the convergence speed and stability.

```
In [5]: learning_rates = [0.001, 0.01, 0.1]
histories = {}

for lr in learning_rates:
    _, _, hist = gradient_descent(X_train_scaled, y_train, learning_rate=lr,
histories[lr] = hist
    print(f'Learning Rate: {lr} -> Final Loss: {hist[-1]:.4f}')
```

Learning Rate: 0.001 -> Final Loss: 5.2583

Learning Rate: 0.01 -> Final Loss: 2.8266

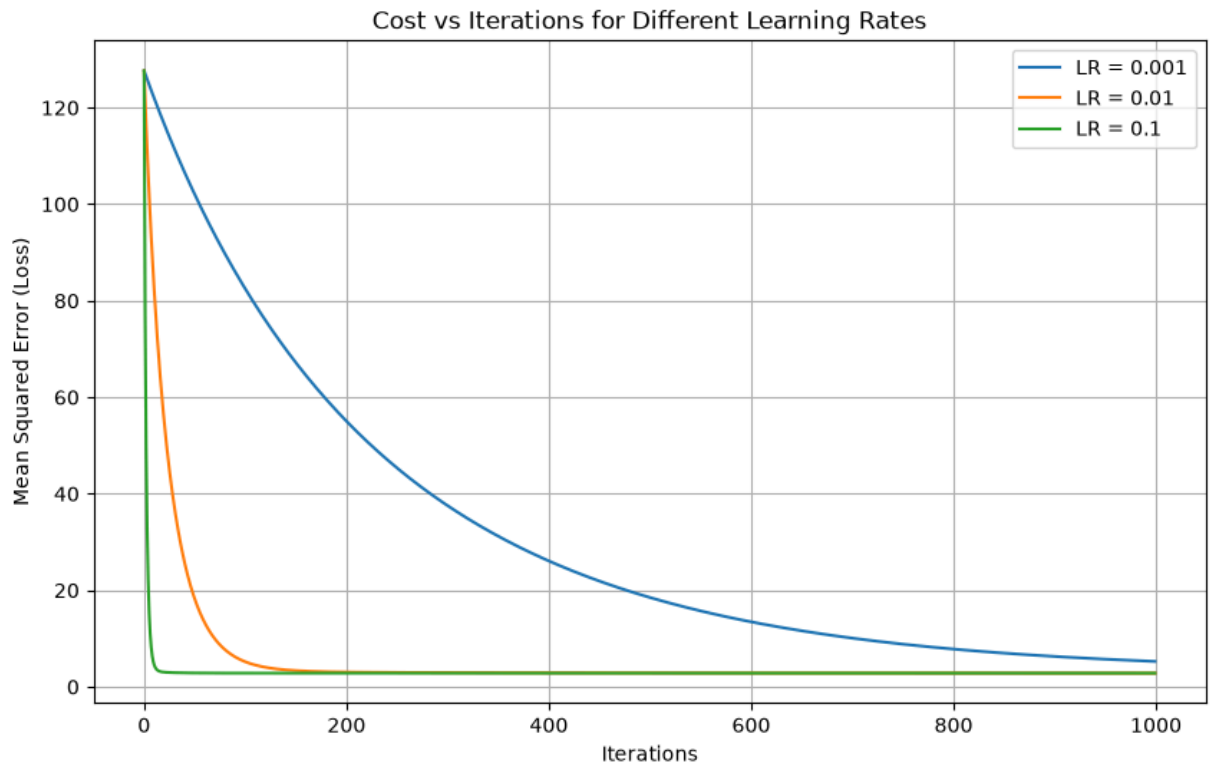
Learning Rate: 0.1 -> Final Loss: 2.8231

Step 7: Plot the Loss versus Iterations

Visualizing the cost function over the number of iterations for the different learning rates.

```
In [6]: plt.figure(figsize=(10, 6))
for lr, hist in histories.items():
    plt.plot(range(1000), hist, label=f'LR = {lr}')

plt.xlabel('Iterations')
plt.ylabel('Mean Squared Error (Loss)')
plt.title('Cost vs Iterations for Different Learning Rates')
plt.legend()
plt.grid(True)
plt.show()
```



Step 8: Evaluate the Model

Calculating MAE, MSE, RMSE, and R^2 Score on the testing dataset using the best learning rate model (0.01).

```
In [7]: y_pred_test = np.dot(X_test_scaled, W) + b

mae = mean_absolute_error(y_test, y_pred_test)
mse = mean_squared_error(y_test, y_pred_test)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_test)

print('Model Evaluation Metrics on Test Data:')
print(f'Mean Absolute Error (MAE): {mae:.4f}')
print(f'Mean Squared Error (MSE): {mse:.4f}')
print(f'Root Mean Squared Error (RMSE): {rmse:.4f}')
print(f'R2 Score: {r2:.4f}')
```

```
Model Evaluation Metrics on Test Data:
Mean Absolute Error (MAE): 1.6557
Mean Squared Error (MSE): 5.6676
Root Mean Squared Error (RMSE): 2.3807
R2 Score: 0.7236
```

Step 9: Interpretation of Results

Based on the evaluation and convergence plots, here are the observations:

1. **Convergence:** A learning rate of 0.1 converges very fast, but is slightly less stable. A learning rate of 0.001 is too slow and doesn't fully converge within 1000 iterations. A learning rate of 0.01 provides a good balance of steady and complete convergence.
2. **Prediction Performance:** The R^2 score indicates how much variance in the final grade (G3) is explained by our features. The MAE and RMSE tell us the average error in predicting the 20-point scale grade. Because we included past grades (G1 and G2) as features, the model performs very well.